

Arquitectura, datos y metodología Scrum

Sistema Integral de Gestión Académica y Alertas Tempranas

FASE

2

SEMANA

7 de 18

SPRINT

3

CORTE

24 SEP

Propósito. Entregar una explicación defendible, trazable y honesta de las decisiones técnicas, del modelo de datos y del trabajo con Scrum/Jira.

ARQUITECTURA	BASE DE DATOS	SCRUM + JIRA
React, Nginx, Express, PostgreSQL, Docker y Cloudflare.	Esquema escolar 0.5.0, integridad, migraciones y trazabilidad.	Sprints quincenales, historias, Review, Retro y evidencia.



1. Estado actual y postura de defensa

La respuesta corta es: estamos en Fase 2, semana 7 y Sprint 3. El sistema ya tiene una experiencia demostrable, pero todavía no es una solución productiva porque la autenticación y los datos principales siguen siendo sintéticos.

PRODUCTO

Prototipo

ESQUEMA

0.5.0

PERSISTENCIA

Pendiente

SPRINT

En curso

ÁREA	QUÉ EXISTE	LÍMITE ACTUAL
Interfaz	Vistas docente y estudiante, login por rol, navegación responsive.	Es un entorno de demostración con datos sintéticos.
API	Express, rutas docentes/estudiantiles, healthcheck y db-health.	La identidad se simula con credenciales y cabeceras demo.
Datos	PostgreSQL 16 y migraciones escolares 001-005.	Los casos de uso aún no usan repositorios persistentes.
Infraestructura	Docker Compose, redes privadas y Cloudflare Tunnel.	La operación debe validarse desde un clon limpio.

Mensaje clave para la comisión

"El prototipo demuestra la dirección del producto. La Fase 2 convierte esa demostración en una vertical segura, persistente y verificable". Esta formulación protege la credibilidad del equipo y muestra que conoce la diferencia entre una maqueta funcional y un sistema listo para producción.

Brecha que gobierna el Sprint 3

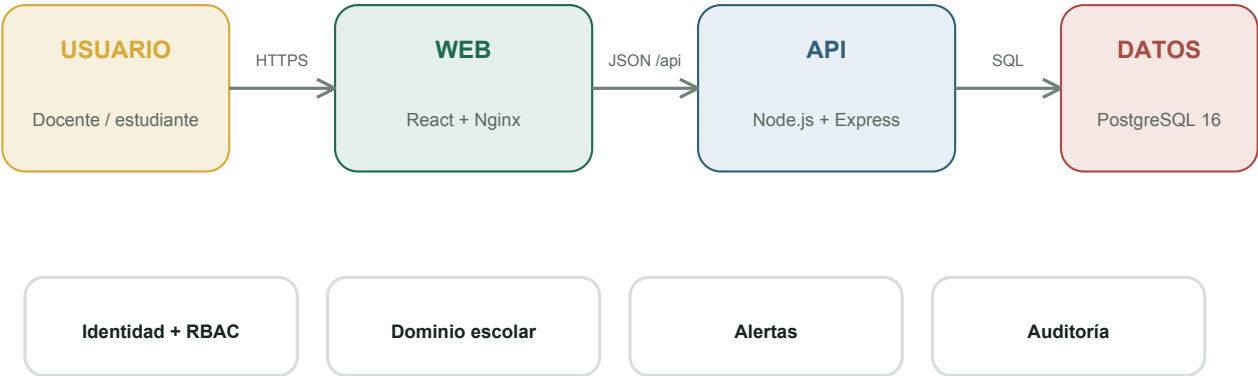
- Reemplazar el login de demostración por autenticación segura.
- Aplicar autorización por rol y alcance en la API.
- Persistir estudiantes y matrículas en PostgreSQL.
- Probar tanto accesos permitidos como denegados.
- Registrar la aceptación o devolución en Sprint Review.



2. Arquitectura de software

SIGAA adopta una arquitectura web modular en tres contenedores. La interfaz presenta la información; la API aplica reglas y permisos; PostgreSQL conserva los datos y su integridad.

ARQUITECTURA MODULAR EN TRES CONTENEDORES



COMPONENTE	RESPONSABILIDAD	NO DEBE HACER
React + Nginx	Navegación, formularios, experiencia responsive y proxy `/api`.	Decidir permisos o ejecutar reglas sensibles.
Express	Casos de uso, validación, autorización, contratos JSON y auditoría.	Depender de estilos o detalles visuales.
PostgreSQL	Relaciones, restricciones, transacciones, consultas e historial.	Exponerse directamente a Internet.
Docker	Versiones reproducibles, redes y healthchecks.	Guardar secretos en la imagen o el repositorio.
Cloudflare	Publicar la web mediante HTTPS sin abrir puertos de la base.	Otorgar acceso administrativo al equipo.

Flujo de una operación

1. El usuario opera la interfaz. 2. Nginx envía `/api` a Express. 3. La API autentica, autoriza y valida. 4. PostgreSQL ejecuta una transacción. 5. La API registra trazabilidad y devuelve JSON. 6. React actualiza la vista.



3. Cómo defender las decisiones técnicas

Monolito modular, no microservicios	Un equipo de tres personas y un semestre necesitan simplicidad operacional. Los módulos pueden separarse después si las métricas lo justifican.	Menos despliegues, menos fallos de red y pruebas más simples.
PostgreSQL, no una base documental	El dominio académico contiene relaciones fuertes, reglas de unicidad y operaciones que deben ser atómicas.	Integridad, transacciones e historial consistente.
Docker como contrato de ejecución	Evita depender de versiones instaladas en cada computador.	Mismo comando, mismas dependencias y healthchecks.
Autorización en backend	Ocultar botones no protege datos. Cada ruta debe validar rol, recurso y alcance.	Seguridad verificable con pruebas negativas.
Alertas explicables	Cada alerta guarda regla, versión y evidencia; no se usa una predicción opaca en el MVP.	Trazabilidad y confianza para decisiones escolares.

Atributos de calidad que se pueden comprobar

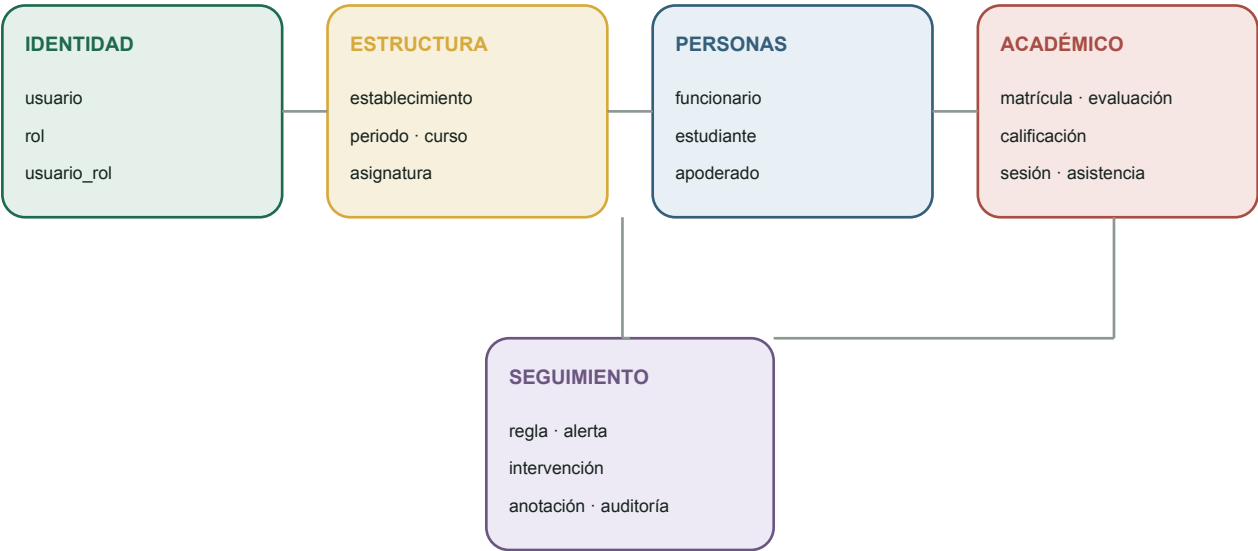
Portabilidad	`docker compose up --build` inicia web, API y base.
Seguridad	Red privada, secretos externos, RBAC y pruebas 401/403.
Disponibilidad	Tres healthchecks y reinicio controlado de servicios.
Mantenibilidad	Módulos, contratos, migraciones y decisiones versionadas.
Explicabilidad	Toda alerta expone regla, umbral y evidencia.



4. Arquitectura de base de datos

El diseño 0.5.0 representa un colegio o liceo y separa identidad, estructura escolar, personas, operación académica y seguimiento.

DOMINIOS DEL MODELO RELACIONAL ESCOLAR 0.5.0



RELACIÓN	SIGNIFICADO	REGLA
Estudiante - Matrícula - Curso	Ubica al estudiante en un curso y periodo.	Evitar duplicidad activa.
Funcionario - Asignación - Curso/Asignatura	Delimita lo que cada profesor puede operar.	Conservar vigencia histórica.
Evaluación - Calificación - Estudiante	Registra resultados publicados.	Nota 1,0-7,0; una por evaluación.
Sesión - Asistencia - Estudiante	Representa una clase y su registro individual.	Un estado por estudiante y sesión.
Regla - Alerta - Intervención	Explica riesgo y documenta acompañamiento.	Evidencia e historial no se eliminan.



5. Integridad, seguridad y evolución de datos

La arquitectura de datos no es solo un diagrama: las restricciones y migraciones convierten las reglas escolares en controles verificables.

1	Identidad	UUID técnico; RUN, RBD, correo y códigos únicos cuando corresponde.
2	Dominio	Notas entre 1,0 y 7,0; ponderaciones válidas; estados controlados.
3	Consistencia	Transacciones para operaciones compuestas y claves foráneas para relaciones.
4	Historial	Vigencias docentes, correcciones y anulaciones con autor, motivo y fecha.
5	Privacidad	La API limita el alcance; estudiante y apoderado solo consultan vínculos propios.
6	Migración	Scripts 001-005 ordenados; `app_metadata` identifica versión y perfil escolar.

Evolución correcta del prototipo

La API ya crea un `Pool` de PostgreSQL y puede verificar la conexión, pero las rutas docentes y estudiantiles siguen leyendo `prototype-data.js`. La transición recomendada es:

- Crear repositorios SQL por agregado.
- Ejecutar operaciones compuestas dentro de transacciones.
- Reemplazar cabeceras demo por una identidad de sesión.
- Agregar pruebas de integración con base temporal.
- Mantener datos sintéticos únicamente como seed de demostración.

Riesgo que evitamos

Conectar la interfaz directamente a PostgreSQL omitiría autorización, validación y auditoría. Toda escritura debe atravesar la API.



6. Metodología Scrum y cómo defenderla

Scrum se adapta al calendario académico mediante sprints quincenales. No se usa para llenar documentos: se usa para inspeccionar un incremento, recibir feedback y ajustar el plan.

ELEMENTO	EN SIGAA	VALOR
Product Owner	Joel prioriza, aclara necesidades y acepta o devuelve historias.	Una sola dirección de producto.
Scrum Master	Facilita eventos, tablero e impedimentos; falta ratificación formal.	Cadencia y transparencia.
Developers	Los tres diseñan, programan, prueban y documentan.	Responsabilidad compartida.
Sprint	Dos semanas con objetivo único y resultado demostrable.	Feedback frecuente.
Definition of Done	Código, prueba, seguridad, documentación, evidencia y aceptación.	Calidad común y verificable.

Por qué Scrum es adecuado

- El problema escolar requiere validación progresiva con usuarios.
- Las entregas Duoc crean hitos, pero los sprints permiten controlar el trabajo entre hitos.
- Arquitectura, seguridad y datos pueden inspeccionarse antes de acumular defectos.
- Las historias incompletas vuelven al backlog; no se ocultan como terminadas.
- Review y Retro separan feedback de producto y mejora del proceso.

Defensa en una frase

"Elegimos Scrum porque necesitamos aprender y entregar durante el semestre, no descubrir al final que construimos algo incorrecto".



7. Jira e historias de usuario

Jira debe mostrar el estado real del trabajo. Cada tarjeta representa una necesidad o una unidad técnica verificable y enlaza la evidencia producida.

NIVEL	EJEMPLO SIGAA	CONTENIDO MÍNIMO
Epic	Seguridad y acceso	Objetivo, alcance y relación con historias.
Story	PB-012 Roles y permisos	Como/quiero/para, criterios, puntos y responsable.
Task	Middleware de autorización	Trabajo técnico, definición de salida y enlace.
Bug	Estudiante consulta ID ajeno	Pasos, esperado, obtenido, severidad y evidencia.

Flujo recomendado

BACKLOG	SELECCIONADA	EN CURSO	EN REVISIÓN	TERMINADA
---------	--------------	----------	-------------	-----------

Reglas de transparencia

- Máximo una historia En curso por integrante.
- Toda Story Terminada enlaza commit, pruebas y evidencia.
- Los puntos no cambian para ocultar retrasos durante el sprint.
- Un Bug crítico impide cerrar la historia relacionada.
- Solo la Review registra la aceptación del Product Owner.

Ejemplo de historia bien formulada

Como estudiante, quiero consultar únicamente mis calificaciones publicadas para conocer mi avance sin acceder a información de otros compañeros.

Criterios: identidad de sesión obligatoria; ID ajeno devuelve 403; solo notas publicadas; prueba positiva y negativa; evidencia enlazada en Jira.



8. Fase y estado de los sprints

Según el cronograma académico, la Fase 2 comenzó en septiembre. La fecha de corte ubica al proyecto en la semana 7, dentro del Sprint 3.

SPRINT	FECHAS	RESULTADO	ESTADO REAL
0	10-22 ago	Visión, backlog, riesgos, DoD, repo y Docker.	Informe listo; falta aceptación formal registrada.
1	24 ago-5 sep	Arquitectura, ER, UML y prototipo.	Construido; Review/Retro requieren cierre formal.
2	7-19 sep	Seguridad y plataforma.	Periodo vencido; PB-011/012 incompletas y arrastradas.
3	21 sep-3 oct	Vertical persistente de estudiantes y matrículas.	Vigente; regularización documental al 24-09.

Sprint Goal vigente

Permitir que un usuario autenticado opere únicamente dentro de su rol y que estudiantes y matrículas se lean y escriban realmente en PostgreSQL.

ID	HISTORIA	PTS	ORIGEN
PB-011	Autenticación segura	8	Arrastre Sprint 2
PB-012	Roles y permisos	8	Arrastre Sprint 2
PB-015	Gestión persistente de estudiantes	8	Plan Sprint 3
PB-016	Matrículas por curso y periodo	5	Plan Sprint 3
TOTAL	Compromiso propuesto	29	Sujeto a ratificación del equipo



9. Historias y trabajo del Sprint 3

PB-011 · Autenticación segura
Como usuario, quiero iniciar una sesión segura para proteger mi información.
Criterios: Hash de contraseña; sesión revocable; secretos externos; errores genéricos.

PB-012 · Roles y permisos
Como establecimiento, quiero controlar acciones por rol y alcance para impedir accesos indebidos.
Criterios: Estudiante solo ve lo propio; profesor solo opera asignaciones; 403 y pruebas negativas.

PB-015 · Gestión de estudiantes
Como personal autorizado, quiero mantener estudiantes vigentes en el sistema.
Criterios: Persistencia PostgreSQL; validación; duplicados controlados; auditoría.

PB-016 · Matrículas
Como encargado académico, quiero asociar estudiante, curso y periodo.
Criterios: Curso/periodo válidos; sin duplicidad activa; historial; persistencia tras reinicio.

Distribución equilibrada propuesta

Joel	Sesiones, rutas protegidas e integración API.
Agustín	Formularios, estados de acceso y pruebas de interfaz.
Matías	Repositorios PostgreSQL, migraciones y pruebas de integridad.
Equipo	Criterios, revisión cruzada, Docker, Review y Retro.



10. Sprint Review y Retrospective

Son eventos distintos. La Review inspecciona el producto y permite al Product Owner aceptar o devolver historias. La Retro inspecciona cómo trabajó el equipo y define mejoras.

SPRINT REVIEW	SPRINT RETROSPECTIVE
Participan: equipo, Product Owner e interesados.	Participa: equipo Scrum.
Pregunta: ¿qué valor funciona y qué feedback cambia el backlog?	Pregunta: ¿cómo podemos trabajar mejor?
Salida: historias aceptadas, devueltas o replanificadas.	Salida: una o dos acciones medibles con responsable y fecha.

Guion de Review - 3 de octubre

- Levantar desde clon limpio con Docker.
- Iniciar sesión como docente y estudiante.
- Mostrar un acceso permitido y otro denegado.
- Crear un estudiante y matricularlo.
- Reiniciar la API y comprobar persistencia.
- Mostrar pruebas, deuda y límites conocidos.
- Registrar la decisión del PO por cada historia.

Acciones propuestas para la Retro

ACCIÓN	MEDICIÓN
Actualizar Jira al terminar cada sesión.	Tarjeta con estado y enlace el mismo día.
Limitar trabajo en curso.	Una historia por integrante.
Integrar dos veces por semana.	Commit o pull request martes y viernes.
Cerrar Review y Retro el mismo día.	Actas registradas el 3 de octubre.



11. Guion oral y respuestas difíciles

Guion sugerido - 90 segundos

"SIGAA utiliza una arquitectura web modular de tres contenedores: React y Nginx para la interfaz, Node.js con Express para las reglas y PostgreSQL para la persistencia. Elegimos esta solución porque separa responsabilidades, es mantenible para un equipo de tres personas y se ejecuta igual en cualquier equipo mediante Docker. La base de datos representa el dominio escolar con integridad relacional y trazabilidad. Trabajamos con Scrum en sprints de dos semanas y administramos épicas e historias en Jira. Hoy estamos en Fase 2, semana 7 y Sprint 3. El prototipo ya demuestra los roles docente y estudiante, pero todavía usa datos sintéticos; por eso la meta vigente es autenticación, permisos y persistencia real. En la Review demostramos el incremento y el Product Owner decide su aceptación; en la Retrospective acordamos mejoras del proceso."

¿Por qué no microservicios?

Añadirían despliegues, red y observabilidad sin valor proporcional para tres personas y dieciocho semanas.

¿Por qué PostgreSQL?

Matrículas, cursos, notas y asistencia requieren relaciones, restricciones, transacciones e historial.

¿Está terminado?

No. Existe un prototipo y una arquitectura ejecutable; la Fase 2 debe convertirlos en una vertical persistente y segura.

¿Qué ocurre si una historia no cumple?

No se marca como Terminada; vuelve al backlog y el Product Owner decide su prioridad.

¿Review y Retro son lo mismo?

No. Review inspecciona el producto con interesados; Retro inspecciona el proceso del equipo.

¿Jira demuestra que trabajaron?

Jira da trazabilidad, pero cada tarjeta debe enlazar código, prueba, documento o evidencia.



12. Plan inmediato de regularización

El objetivo es recuperar transparencia de proceso sin inventar ceremonias pasadas y proteger el incremento que debe demostrarse el 3 de octubre.

CUÁNDO	ACCIÓN	EVIDENCIA
24 sep	Ratificar Sprint Goal, responsables y 29 puntos.	Planning y tablero Jira.
25-26 sep	Implementar sesión y autorización base.	Pruebas login, 401 y 403.
27-29 sep	Persistir estudiantes y matrículas.	SQL, API y pruebas de integración.
30 sep-1 oct	Integración web/API y casos negativos.	Demo y capturas.
2 oct	Congelar candidata y ejecutar Docker limpio.	Checklist y hash de versión.
3 oct	Realizar Review y Retrospective.	Decisiones PO y acciones de mejora.

Checklist antes de defender

- Jira coincide con el estado real del repositorio.
- No se presenta login demo como autenticación segura.
- La base no se expone públicamente.
- La demo conserva datos después de reiniciar la API.
- Cada integrante puede explicar arquitectura, datos y Scrum.
- Review y Retro tienen acta, responsables y fecha.

Conclusión

La defensa sólida no depende de aparentar que todo está terminado. Depende de mostrar decisiones coherentes, evidencia verificable, límites conocidos y un plan de Sprint 3 que cierre la brecha entre prototipo y producto.

Fuentes internas: arquitectura de contenedores, modelo ER 0.5.0, Product Backlog, ADR-001/002, Docker Compose, migraciones 001-005 y documentación Scrum versionada en el repositorio SIGAA.